

Reviewer Pack — Minimal Local Induction Dimension of Boolean Functions and C...

Entry d08b98fde951 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 13:36:39 UTC

Minimal Local Induction Dimension of Boolean Functions and Communication Complexity Rank Variance

Entry ID: d08b98fde951 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 13:36:39 UTC

1. Conjecture

Field A (mathematical branch): Descriptive Set Theory (Local Induction Dimension) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

The minimal local induction dimension (mild) of a Boolean function f is linearly correlated with its communication complexity rank variance ($\text{rcv}(f)$), such that $\text{mild}(f) = \Theta(\text{rcv}(f))$ for all n -input Boolean functions.

Rationale (proposer's reasoning):

Local induction dimension captures the complexity of approximating a given function, which could provide insights into the complexity of evaluating functions in communication complexity. The correlation might reveal underlying structures or regularities in the evaluation process that are not immediately apparent.

Taxonomy category: challenge (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 52000701a43d344a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient (r) between $\text{mild}(f)$ and $\text{rcv}(f)$ for all n -input Boolean functions exceeds 0.5, with $p\text{-value} \leq 0.05$, across at least 100 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "local induction dimension" AND "communication complexity" AND "Boolean function" - "mild of Boolean functions" AND "rank variance" AND "Descriptive Set Theory" - "minimal local induction dimension" ~ "0(rank variance)" AND "Communication Complexity Rank Variance"

Top relevant hits considered: - [http://arxiv.org/abs/2001.04131v1] Observation of a resonant structure in $e^+e^- \rightarrow K^+K^-\pi^0\pi^0$ - [http://arxiv.org/abs/physics/0604022v1] A Qualitative Description of Boundary Layer Wind Speed Records - [http://arxiv.org/abs/1007.4650v1] Orbital Selective Pressure-Driven Metal-Insulator Transition in FeO from Dynamical Mean-Field Theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
import math
import sys

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def minimal_local_induction_dimension(f):
        n = int(math.log2(len(f)))
        count_0 = sum(1 for x in f if x[0] == 0)
        count_1 = sum(1 for x in f if x[0] == 1)
        return max(count_0, count_1) / len(f)

    def communication_complexity_rank_variance(f):
        n = int(math.log2(len(f)))
        rank = 0
        for i in range(n):
            count_0 = sum(1 for x in f if x[i] == 0)
            count_1 = sum(1 for x in f if x[i] == 1)
            rank += max(count_0, count_1)
        return (rank / n) ** 2

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
```

```

    f = generate_boolean_function(n)
    mild = minimal_local_induction_dimension(f)
    rcv = communication_complexity_rank_variance(f)
    results.append((mild, rcv))

n_max = max(n for _, _ in results)
instances_tested = len(results) * len([5, 10, 15, 20, 30, 40])

if n_max < 16:
    return {
        "metric_name": "mild vs rcv",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "n_max < 16"
    }

mild_values = [mild for mild, _ in results]
rcv_values = [rcv for _, rcv in results]

mean_mild = sum(mild_values) / instances_tested
mean_rcv = sum(rcv_values) / instances_tested

def pearson_correlation(x, y):
    n = len(x)
    sum_x = sum(x)
    sum_y = sum(y)
    sum_xy = sum(xi * yi for xi, yi in zip(x, y))
    sum_xx = sum(xi ** 2 for xi in x)
    sum_yy = sum(yi ** 2 for yi in y)
    numerator = n * sum_xy - sum_x * sum_y
    denominator = math.sqrt((n * sum_xx - sum_x ** 2) * (n * sum_yy - sum_y ** 2))
    return numerator / denominator if denominator != 0 else 0

r = pearson_correlation(mild_values, rcv_values)

return {
    "metric_name": "mild vs rcv",
    "metric_value": r,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": r > 0.5,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2*i + 1 for i in range(5, 6)] # Default to a list of 30 primes

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{trial_result} }}")
        results.append(trial_result)

```

```

mean_r = sum(result["metric_value"] for result in results if result["metric_value"] is not None) / len(results)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["metric_value"] is not None for result in results):
    RESULT = "SUPPORTED" if support_fraction >= 0.8 else "FALSIFIED"
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    RESULT = f"FALSIFIED counterexample=\"first_failing_seed={first_failing_seed}\""
else:
    RESULT = "INCONCLUSIVE insufficient_data"

print(f"RESULT: {RESULT} mean={mean_r:.2f} std=0.00 support_fraction={support_fraction:.2f}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_610864ad.py", line 94, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_610864ad.py", line 42, in run_trial
    mild = minimal_local_induction_dimension(f)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_610864ad.py", line 26, in minimal_local_induction_dimension
    count_0 = sum(1 for x in f if x[0] == 0)
               ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_610864ad.py", line 26, in <genexpr>
    count_0 = sum(1 for x in f if x[0] == 0)
                   ~^^
TypeError: 'int' object is not subscriptable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it was unable to compute the Pearson correlation coefficient and p-value as required by the support condition. | next: Investigate the cause of the crash in the test code and attempt to run the test again to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13564
2	propose	ollama_remote	glm4:latest	0	0	11681
3	preregistration	ollama_remote	glm4:latest	0	0	9503
4	novelty	ollama_remote	glm4:latest	0	0	8943
5	novelty	ollama_remote	glm4:latest	0	0	10075
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13791
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10668
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10195
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12099
10	judge	ollama_remote	glm4:latest	0	0	37299

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 137818 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/d08b98fde951.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d08b98fde951.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d08b98fde951.tar.gz` (if generated)